

Регистрация метаданных

История изменений технической документации

Что было изменено	Кто изменил	Когда
По проекту добавлено описание критичных и отложенных регистраций, а также раздел по обновлению регистратора при невозможности чтения дерева проекта.	Кузнецов А.П.	20.06.2023
Описание процесса создания новых регистраций вынесено в отдельные документы.	Дмитерко С.И.	28.11.2024

Термины и определения

Metadata-registrator - приложение и сервис в составе этого приложения, содержащий в себе логику разбора метаданных дистрибутива, и производящий регистрацию метаданных в целевом сервисе.

Целевой сервис - middleware-сервис в составе облака, принимающий данные от сервиса регистрации в момент обновления приложений.

Гибкий способ регистрации - в данном варианте разработчик middleware-сервиса заводит на сервисе регистрации собственный модуль регистрации и реализует в нем логику разбора дистрибутива и передачи данных на свой сервис.

Упрощенный способ регистрации - данный способ не требует реализации модуля регистрации, а лишь добавление описания нового типа регистрации для ExtCollector и реализацию методов приема данных регистрации со строго заданной сигнатурой.

Модуль регистрации - модуль на сервисе регистрации, отвечающий за сбор и регистрацию данных для определенного сервиса приемника. Отвечает за модуль разработчик middleware-сервиса.

ExtCollector - особый модуль регистрации, позволяющий собирать данные из дистрибутива в заранее определенном формате json.

Критичность регистрации - признак регистрации, указывающий на то, что без выполнения регистрации с этим признаком сервис может не запуститься, либо работать некорректно.

Концептуальное описание решения и основные алгоритмы

Функционал всех регистраторов выделен в приложение metadata-registrator. Регистраторы можно реализовывать двумя способами: гибким или упрощенным.

В гибком варианте в отдельных модулях реализована каждая из регистраций таким образом, чтобы сам Хоттабыч ничего не знает о специфике их регистрации. Ответственность за разработку и поддержку этих модулей находится на разработчике middleware-сервиса, в котором производится регистрация. Модули регистрации описывают определенные методы с фиксированной сигнатурой для сбора данных из дистрибутива и вызова самих регистраций, а Хоттабыч в нужные точки фаз обновления в нужном порядке их вызывает. Порядок регистраций выставляется зависимостями между модулями регистрации. Для упрощения разработки модулей регистрации разработаны методы-хелперы, обходящие нужные файлы дистрибутива, таким образом можно реализовать модуль используя только код на Python.

Упрощенный режим требует меньшего участия прикладного разработчика в процессе внедрения регистрации. В этой реализации ему лишь требуется описать формат файлов в специальном модуле платформы и реализовать определенный набор методов на сервисе, принимающем регистрационные данные. В данном режиме парсинг дистрибутива, слияние данных из модулей, осуществление вызовов на сервис и сохранение состояния между фазами сервис регистраторов берет на себя.

Для стабильности работы механизма регистраций в случае временной недоступности сервисов, принимающих регистрацию, реализован механизм критичных/отложенных регистраций, он позволяет дожидаться их прохождения. При любом типе регистрации в случае недоступности/получения ошибки невозможности выполнить метод на запись

регистратор повторно пытается эту регистрацию провести, пока она не выполнится. А разделение на критичные/отложенные регистрации позволяет разделить и ответственность регистрирующих сервисов в разрезе их простоя: если это некритичный сервис, то даже если он достаточно продолжительное время будет недоступен - это не повлияет на ход выполнения работ по выполнению обновлений других приложений. Если же это критичный сервис, то его недоступность лишь на время задержит проведение регистраций (и соответственно самих обновлений) других приложений.

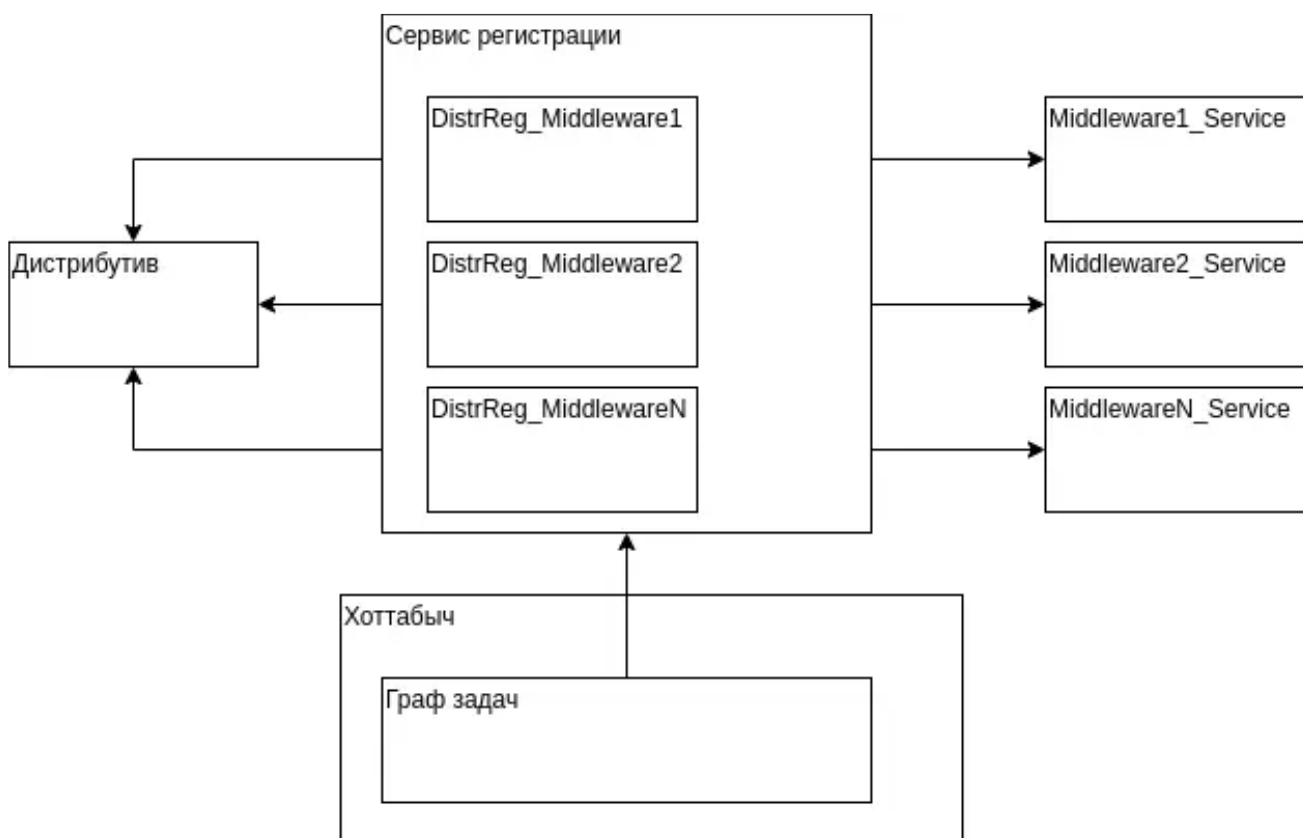
Архитектура продукта

Подсистемы, сервисы, базовые сущности и их взаимосвязи

Модули регистрации

Работа всего функционала регистрации данных выделена в сервис регистрации в составе приложения metadata-registrator. Это позволяет обновлять функционал регистрации независимо от релизного цикла Хоттабыча, а также не заваливать Хоттабыч долгими запросами регистрации.

В базовом модуле сервиса содержатся api-методы сервиса и методы-хелперы, упрощающие сбор данных по дистрибутиву.



Хоттабыч при запуске обновления запрашивает у сервиса регистраторов множество объектов с префиксом "DistrReg_" и по их именам создает соответствующее количество логик для регистрации. Для исключения логик неработоспособного сервиса из всех запускаемых обновлений вводится настройка облака "Управление версиями.Обновления.ЧерныйСписокРегистраторов", содержащий системные имена middleware-сервисов, логики для которых не должны создаваться в обновлениях. Если между модулями регистрации были настроены зависимости, то эти зависимости также переносятся на порядок исполнения задач регистрации на фазах обновлений.

На фазе подготовки Хоттабыч вызывает с сервиса регистрации метод MetadataRegistrar.ParseDistribution, передавая на вход имя дистрибутива и версию. Сервис регистраторов смотрит в своей БД, есть ли сохраненные данные для этой версии дистрибутива. Если есть - возвращает данные Хоттабычу не скачивая повторно дистрибутив. Если нет - скачивает дистрибутив из хранилища, распаковывает его на диске, вычитывает данные, необходимые Хоттабычу для обновления и из каждого объекта регистратора вызывает метод CollectData для каждого регистратора. Собранные данные для регистрации сохраняются в таблицу DistrRegData, состоящую из полей:

- VersionId: Int64 - идентификатор версии дистрибутива;
- RegName: String - название логики регистратора;
- RegData: String - данные для регистрации из дистрибутива.

При наступлении момента вызова подготовительной фазы регистрации, Хоттабыч для каждого объекта регистрации вызывает метод MetadataRegistrar.Prepare, дожидаясь успешного завершения работы метода. В случае падения

одного из регистраторов с ошибкой, соответствующая логика выделяется красным цветом и обновление падает с ошибкой. Метод Prepare осуществляет удаленный вызов на сервис, принимающий данные и возвращает статус регистрации в Хоттабыч.

На фазах обновления, отката и подтверждения аналогичным образом для каждой логики регистрации вызываются методы Update, Rollback, Clean с аналогичной сигнатурой соответственно.

Упрощенный режим регистрации

Для работы с упрощенными видами справочников регистрации используется модуль ExtCollector из состава в SDK платформы. При загрузке этот модуль ищет файлы с описанием форматов всех существующих видов справочников для регистрации (файлы *.extregistartion). В процессе работы у него вызывается метод ExtCollector.Get/2, выполняющий сбор и слияние данных в один json объект из указанного места файловой системы на основании информации ранее загруженной из справочников.

Сервис регистрации в методе MetadataRegistrar.ParseDistribution кроме анализа всех описанных модулей регистрации также анализирует файлы форматов в модуле ExtCollector. Все форматы, считанные из файлов *.extregistartion" добавляются в Хоттабыч в качестве логик регистрации так же, как и модули регистрации.

В момент разбора дистрибутива через API модуля ExtCollector происходит сбор данных по всем форматам, подлежащим регистрации.

Методы MetadataRegistrar.Prepare, Update, Rollback, Clean в упрощенном режиме, в отличие от реализации модулей регистрации, вызывают не методы из модулей регистрации, а собственную реализацию вызовов регистрации по сценарию задач планировщика с сохранением состояния на стороне сервиса регистрации.

Сценарий вызовов регистрации в упрощенном режиме

В методы регистрации первым параметром передается название целевого объекта, по нему сервис регистрации определяет сервис для вызова методов:

- Сервис регистрации проверяет, нужно ли вообще проводить регистрацию. Для этого проводится анализ наличия регистрируемых данных через ExtCollector, и если этих данных нет - на целевом сервисе вызывается метод DataExists. Если этот метод вернет, что по этому приложению никаких данных ранее не регистрировалось - дальнейшие вызовы по данному регистратору не выполняются.
- В методе MetadataRegistrar.Prepare получает список данных необходимых для deregistrации - вызывает метод GetUnregister и сохраняет этот список в таблице SavedState.
- В методе MetadataRegistrar.Update происходит регистрация нового списка - вызов метода Register.
- В методе MetadataRegistrar.Rollback - вызов метода Register для старого (сохраненный в SavedState) и метода PrepareUnregister для нового списка (из дистрибутива).
- В методе MetadataRegistrar.Commit - вызов метода Unregister со старым списком в случае успешного обновления или с новым в случае отката обновления.

Обновления в других облаках

Если обновление приложения происходит в облаке, отличном от облака текущего Хоттабыча, то все вызовы на сервис регистраторов осуществляется в то облако, где находится обновляемое приложение. Сам сервис регистраторов осуществляет вызовы на middleware-сервисы внутри того же облака просто по системному имени.

Сохранение состояния

Для сохранения состояния модуля между фазами обновлений использоваться отдельная таблица "SavedState". Для облегчения работы с сохранением состояния существуют методы-хелперы RegistrationHelpers.SaveState, RegistrationHelpers.GetSavedState, позволяющие для текущей регистрации как сохранить, так и получить ранее сохраненные данные.

Ограничения для middleware-сервисов, производящих регистрацию

Так как каждое обновление приложения в Хоттабыче вызывает цепочку регистраций данных в middleware-сервисах, недоступность одного из этих сервисов блокирует работу всех обновлений. Поэтому к сервису, производящему регистрацию, предъявляются повышенные требования к стабильности его работы.

В процессе обновления самого сервиса тоже происходит регистрация, в том числе в самом себе. Из-за этого нежелательны обновления этих сервисов в блокирующем режиме. Подойдет, допустим, легкая конвертация. В случае, если без полного обновления не обойтись - этот сервис должен обновляться отдельно ото всех и в таком случае для него регистрация данных произойдет до блокировки самого сервиса.

Сценарий разработки нового модуля регистрации

Процесс описан в соответствующих разделах ТД:

- [Создание нового модуля регистрации;](#)

- [Критичная регистрация;](#)
- [Отложенная регистрация;](#)
- [Упрощенная регистрация.](#)

Базовый вариант API middleware-сервиса регистратора на примере feature-ctrl

Модуль feature-регистратора собирает данные для регистрации по feature-файлам, находящимся во всех модулях всех сервисов: UI и БЛ.

1. На фазе подготовки в методе Prepare модуль регистрации не делает ничего.
2. На фазе обновления в методе Update модуль вызывает с сервиса feature-ctrl метод FeatureRegistrar.Prepare, передающий сервису данные для регистрации. Использовать сервис их еще не начинает.
3. На фазе отката (в случае если решили отказаться от обновления), метод Rollback вызывает с feature-ctrl метод FeatureRegistrar.Rollback, откатывающий внесенные изменения.
4. На фазе подтверждения в случае успешного прохождения обновления метод Clean вызывает с feature-ctrl метод FeatureRegistrar.Commit, подтверждающий использовать новые данные и не использовать старые.

Обновление metadata-registrator при невозможности чтения дерева проекта

Реализован особый режим обновления метадата-регистратора в случае ошибки чтения его дерева проекта. В этом случае:

- 1) регистратор на разбор дистрибутива отвечает Хоттабычу особым сообщением, содержащим лишь самые необходимые данные для проведения его обновления.
- 2) Хоттабыч пропускает все логики регистрации. Проводит обновление БЛ и базы (если требуется).
- 3) в конце фазы обновления Хоттабыч на регистраторе зовет новый метод, проводящий пакетную регистрацию. В этом методе проводится сразу всё: и разбор дистрибутива, и вызов регистраций. Он старается выполнить всё по-максимуму. Если что-то не выходит - выдает исключение с описанием того, что не получилось и с предложением провести повторное легкое обновление вручную.

Такое решение позволяет не создавать костыли на сборке, хранилище и не вмешиваться в процесс обновления других сервисов.

Критичные/отложенные регистрации

С критичными регистрациями всё тривиально - регистратор делает несколько попыток регистраций, пока они не выполнятся. В случае достижения таймаута ожидания критичной регистрации обновление падает с ошибкой и не дает продолжить обновление узлов без их завершения.

Рассмотрим подробнее отложенные регистрации. Хоттабыч при вызове методов передачи регистрационных данных на фазах обновления кроме существующих параметров, теперь передает еще и идентификатор обновления, он потребуется в дальнейшем для отслеживания отложенных задач.

В БД метадата-регистратора в таблице RegistrationQueue, хранится очередь регистраций. Она содержит следующую информацию:

1. State - состояние задачи (в очереди, отправлено, ошибка, выполнено);
2. StartTime - дата и время постановки задачи;
3. RegistratorName - название регистрации;
4. AppName - название приложения;
5. DistrName - название дистрибутива;
6. Version - версия;
7. Services - список сервисов;
8. Callback - данные для обратного вызова;
9. Method - название метода регистрации.

При запуске методов Prepare, Update, Rollback, Commit регистратор размещает задачу в очередь. При этом сразу же отдает в Хоттабыч статус, что регистрация отложена.

Хоттабыч этот статус приравнивает к тому, что регистрация выполнена для расчета факта завершения фазы. Это означает, что обновление приложения может завершиться не дожидаясь выполнения отложенных задач. Тем не менее в таблице ИсторияОперацийПродукта присутствует флаг, сигнализирующий наличие незавершенных отложенных регистраций. Этот флаг рассчитывается автоматически задачей планировщика, контролирующей активные

обновления. При наличии этого флага обновление размещается в разделе "Отложенные регистрации" в реестре обновлений.

Задача планировщика сервиса регистрации контролирует ход обработки очереди задач отложенных регистраций. Задачи, находящиеся в очереди - запускаются, отправленные - ждут ответа, ошибочные - пытаются несколько раз повторно запуститься до определенного времени со времени постановки задачи. Выполненные успешно - пропускаются.

Хоттабыч по отложенным регистрациям периодически забирает статусы, у ошибочных - вместе с сообщениями об ошибках. Если все отложенные задачи в обновлении выполнены - у обновления снимается метка наличия отложенных задач и задача при этом исчезает из реестра обновлений. Если приходит понимание, что дальше регистрация отложенных задач не продвинется - на обновлении появляются кнопка возобновления и принудительного завершения обновления.

Схема базы банных

Схема сущностей



Рисунок 1. Схема сущностей регистрации метаданных

Таблицы

Описание таблиц находится [в разделе "Регистрация метаданных"](#) в описании API на сайте [wi.sbis.ru](#).

Типовые выборки

1) Получение идентификатора версии дистрибутива.

Для получения информации о дистрибутиве, данных регистрации или сохраненного состояния пытаемся получить идентификатор версии дистрибутива.

Алгоритм выборки:

Обращаемся к таблице DistrVersion и ищем запись соответствующую названию и версии дистрибутива.

2) Получение метаданных регистратора для версии дистрибутива.

Требуется получить данные определенного регистратора для версии дистрибутива.

Алгоритм выборки:

Обращаемся к таблице DistrRegData и отбираем запись, соответствующую идентификатору версии дистрибутива и названию регистратора.

3) Получение сохраненных при обновлении данных.

Для выполнения регистрации требуется получить промежуточные данные, сохраненные на предыдущем этапе обновления.

Алгоритм выборки:

Обращаемся к таблице SavedState, выполняем фильтрацию по идентификатору версии дистрибутива, названию регистратора и названию приложения.

Индексы

Индекс	Выборки
DistrVersion(Name, Version)	1
DistrRegData(DistrVersionId)	2
DistrRegData(DistrVersionId, RegName)	2
SavedState(DistrVersionId)	3

Публичное API

Не содержит публичного API, с сервисом взаимодействует только Хоттабыч.

Организация кода

Модули, их зависимости

Сервис состоит из модуля MetadataRegistrar, содержащего общий функционал регистраций, а также из набора прикладных модулей с префиксом в имени "DistrReg_" - в этих модулях содержится прикладная логика регистраций.

Репозитории GIT

Код приложения располагается в репозитории: <https://git.sbis.ru/middleware/metadata-registration>

Диаграмма классов

Не требуется, реализация на основе методов БЛ.

Подсистема распределения прав

Методы сервиса доступны только администратору Хоттабыча для осуществления проведения регистраций в других облаках.